Online Casino Simulator

План за разработка на софтуерния проект

Тип проект: Full-stack уеб приложение
Продължителност на разработката: 12 седмици (ориентировъчно)
Архитектура: Модулен монолит (Modular Monolith)
Основни технологии: Java, Spring Boot, React, TypeScript, PostgreSQL, Docker
Версия на документа: 1.1 (29.10.2026) — актуализирана спрямо приетите архитектурни решения; промените са описани в Приложение А
Състояние на проекта: Седмица 1 — завършена; Седмица 2 — планирана

> **Означения:** **Реализирано** — налично в кода на проекта; Планирано (Седмица N) — предстои според графика.

---

1. Обща информация за проекта

1.1. Цел на проекта

Разработване на уеб платформа, която симулира функционирането на реално онлайн казино, като възпроизвежда автентичните правила, вероятности и резултати от печалби и загуби, но без използване на реални пари.

Потребителите няма да могат да депозират, теглят или печелят реални парични средства.

Платформата ще предоставя и образователно съдържание, съобразено с личните интереси на потребителите. Чрез изпълнение на образователни дейности те ще имат възможност да получават допълнителна виртуална валута.

Приложението ще включва и механизми за отговорна игра (Responsible Play), които ще позволяват на потребителите да следят своето поведение и доброволно да ограничават времето, прекарано в платформата.

Важно: Първоначално продуктът ще бъде представян като симулатор на казино и образователна платформа, а не като доказано средство за лечение на хазартна зависимост. Подобни медицински или терапевтични твърдения биха изисквали независимо изследване и професионално потвърждение.

1.2. Целева аудитория

- Пълнолетни потребители, които търсят безплатно забавление с казино игри.
- Хора, които се интересуват от математиката и вероятностите в казино игрите.
- Потребители, които предпочитат симулации без финансов риск от залози с реални пари.
- Хора, които желаят да съчетават образователни дейности със забавление.

1.3. Основни функционалности на MVP

MVP (Minimum Viable Product) представлява първата минимално завършена и използваема версия на приложението.

Основните функционалности ще бъдат:

1. Регистрация и автентикация на потребители.
2. Потребителски профил и избор на образователни интереси.
3. Виртуален портфейл с история на транзакциите.
4. Напълно функционираща европейска рулетка.
5. Библиотека с образователни видеа и тестове.
6. Получаване на виртуални награди чрез образователни дейности.
7. Лично табло със статистика за игрите.
8. Ограничения на игровите сесии и доброволни почивки.
9. Административен панел.

1.4. Функционалности извън обхвата на MVP

Следните функционалности няма да бъдат разработвани в първата версия:

- Транзакции с реални пари или закупуване на виртуална валута.
- Криптовалути или награди, които могат да бъдат обменени за реални пари.
- Мултиплейър казино игри.
- Blackjack, Poker и Slot Machines.
- Игри с крупиета на живо.
- Нативни мобилни приложения.
- Платени AI системи за препоръки.
- Класации и състезателни механизми, насърчаващи прекомерна игра.

2. Използвани технологии

Компонент	Технология	Статус
Frontend	React + TypeScript + Vite	**Реализирано**
Маршрутизация (frontend)	React Router	Планирано (Седмица 2)
Стилизиране	Tailwind CSS	**Реализирано**
Визуализация на игри	Phaser	Планирано (Седмица 6)
Backend	Java 21 + Spring Boot	**Реализирано**
Сигурност	Spring Security	Планирано (Седмица 2)
Сесии	Spring Session JDBC (PostgreSQL)	Планирано (Седмица 2)
API договор и документация	OpenAPI 3.1 (`openapi.yaml`, design-first), springdoc-openapi / Swagger UI	**Реализирано**
База данни	PostgreSQL	**Реализирано**
Работа с базата данни	Spring Data JPA	**Реализирано** (конфигурирано)
Миграции на базата данни	Flyway	**Реализирано**
Локална среда	Docker Compose	**Реализирано**
Backend тестване	JUnit Jupiter (JUnit 6 — наследник на JUnit 5, управляван от Spring Boot), Mockito, Testcontainers	**Реализирано**
Архитектурни тестове	ArchUnit	Планирано (Седмица 2)
Frontend тестване	Vitest, React Testing Library	**Реализирано**
End-to-End тестване	Playwright	Планирано (Седмица 11)
CI/CD	GitHub Actions	**Реализирано (CI)**
Frontend хостинг	Cloudflare Pages	Планирано (Седмица 12)
Backend хостинг	Render	Планирано (Седмица 12)
Хостинг на база данни	Neon	Планирано (Седмица 12)

Първоначално приложението ще бъде разработено като един Spring Boot проект, разделен на логически независими модули.

Този архитектурен подход ще позволи по-лесно разработване, тестване и поддръжка на системата.

Използването на Microservices, Kubernetes, RabbitMQ или Redis не е необходимо за първата версия.

---

3. Архитектура на системата

Приложението ще използва клиент-сървърна архитектура (Client-Server Architecture).

3.1. Frontend — React

Frontend частта ще отговаря за потребителския интерфейс и взаимодействието с платформата.

Основни компоненти:

- Страници за регистрация и вход.
- Начална страница на казиното.
- Интерактивна рулетка.
- Интерфейс на виртуалния портфейл.
- Библиотека с образователни материали.
- Потребителско табло със статистика.
- Настройки за отговорна игра.

3.2. Backend — Spring Boot

Backend частта ще реализира основната бизнес логика на приложението.

Основни модули:

- Автентикация и авторизация.
- Управление на потребителските профили.
- Управление на виртуалните портфейли и транзакции.
- Логика на казино игрите.
- Управление на образователното съдържание.
- Система за образователни награди.
- Ограничения на игровите сесии.
- Генериране на статистики и справки.
- Административни функции.

3.2.1. Организация на модулите и API договор

Всеки backend модул използва еднаква слоеста структура: `controller` (REST), `service` (бизнес логика), `repository` (достъп до данни), `entity` (JPA), `dto` (заявки и отговори), `config` и `exception`. Модулите комуникират единствено чрез своите `service` и `dto` пакети. Правилата ще се проверяват автоматично с ArchUnit (планирано, Седмица 2).

REST API се проектира първо (design-first) във файла `openapi.yaml`, който е единственият източник на истина. Реализацията се проверява автоматично спрямо договора **(реализирано)**.

3.3. База данни — PostgreSQL

Базата данни ще съхранява:

- Потребителски акаунти.
- Баланси на виртуалните портфейли.
- История на транзакциите.
- Казино игри и направени залози.
- Образователни материали.
- Резултати от тестове и награди.
- Потребителски предпочитания.
- Ограничения на игровите сесии.
- Одитни записи.

Основен архитектурен принцип: Backend системата ще генерира всички резултати от игрите, ще изчислява печалбите и ще актуализира виртуалните баланси.

Frontend приложението единствено ще изпраща действията на потребителя и ще визуализира потвърдените от сървъра резултати.

3.4. Сигурност (планирано, Седмица 2)

- Автентикация чрез сесии на сървъра (Spring Security и Spring Session JDBC в PostgreSQL), без токени в браузъра.
- Session cookie с атрибути HttpOnly и SameSite; в production — Secure и префикс `__Host-`.
- CSRF защита за всички заявки, променящи състояние; хеширане на паролите с BCrypt.
- Роли PLAYER и ADMIN; ограничаване на опитите за вход и регистрация; стандартизирани съобщения за грешки (RFC 9457 Problem Details).

---

4. План за разработка

Етап 1 — Основи на проекта (Седмици 1–2)

Цел: Изграждане на архитектурата, средата за разработка и системата за потребителски акаунти.

Седмица 1: Инициализация на проекта

Статус: **Завършена**

Задачи:

1. Създаване на GitHub repository.
2. Инициализиране на React + TypeScript frontend.
3. Инициализиране на Spring Boot backend.
4. Конфигуриране на Docker Compose с PostgreSQL.
5. Изграждане на модулна структура на backend приложението.
6. Конфигуриране на Spring Data JPA и Flyway.
7. Дефиниране на environment variables.
8. Конфигуриране на комуникацията между frontend и backend.
9. Създаване на първоначален GitHub Actions workflow.
10. Документиране на архитектурата и основните потребителски сценарии.

Седмица 2: Автентикация и управление на потребители

Статус: Планирана — детайлен план: `docs/plans/week-02.md` (ревизия 3)

Задачи:

1. Проектиране на модела за потребители в базата данни.
2. Реализиране на регистрация.
3. Добавяне на сигурно хеширане на паролите.
4. Реализиране на вход и изход чрез защитени session cookies.
5. Добавяне на автентикация и авторизация със Spring Security.
6. Дефиниране на потребителски роли: PLAYER и ADMIN.
7. Разработване на потребителски профил.
8. Добавяне на избор и редактиране на интереси.
9. Валидиране на входните данни и стандартизиране на API грешките.
10. Написване на тестове за автентикацията.
11. Проектиране на API договора за автентикацията и профила в `openapi.yaml` преди реализацията.
12. CSRF защита за всички заявки, променящи състояние, с обновяване на токена след вход и изход.
13. Ограничаване на опитите за вход и регистрация, без възможност за злонамерено заключване на чужди акаунти.

14. Ограничение на сесиите: 30 минути неактивност и максимална продължителност от 12 часа.
15. Автоматични тестове на архитектурните правила и на съответствието с API договора.

Съгласно design-first подхода задача 11 се изпълнява преди реализацията на endpoints (задачи 2–8).

Очакван резултат: Потребителите могат да се регистрират, да влизат сигурно в системата и да управляват своите профили.

Етап 2 — Виртуална валута и игрова логика (Седмици 3–4)

Цел: Разработване на основната бизнес логика, преди да бъдат добавени визуалните анимации на игрите.

Седмица 3: Виртуален портфейл

Задачи:

1. Създаване на Wallet и Transaction entities.
2. Автоматично предоставяне на начален виртуален баланс на всеки нов потребител.
3. Реализиране на функционалност за извличане на текущия баланс.
4. Реализиране на операции за добавяне и изваждане на виртуална валута.
5. Записване на всички транзакции в непроменим регистър (Immutable Ledger).
6. Предотвратяване на отрицателни балансы.
7. Използване на database transactions и заключване на записи, когато е необходимо.
8. Добавяне на уникални idempotency keys за предотвратяване на дублирани операции.
9. Написване на тестове за едновременни заявки и коректност на транзакциите.

Всички баланси ще се съхраняват като цели числа, представящи виртуални монети, вместо да се използват floating-point стойности.

Седмица 4: Логика на рулетката

Задачи:

1. Дефиниране на правилата на европейската рулетка.
2. Реализиране на 37 възможни резултата: от 0 до 36.
3. Използване на криптографски сигурен генератор на случайни числа на сървъра.

4. Реализиране на различни видове залози:
    - Залог на конкретно число.
    - Червено или черно.
    - Четно или нечетно.
    - Ниски или високи числа.
    - Залози на дузини.
5. Проверка на наличния баланс и лимитите на залозите.
6. Изчисляване на печалбите според стандартните правила.
7. Записване на игровите рундове и индивидуалните залози.
8. Интегриране на рулетката с виртуалния портфейл.
9. Написване на Unit Tests за изчисляването на печалбите и специфичните случаи.

Очакван резултат: Напълно функциониращ backend, който приема виртуални залози, генерира резултати от рулетката, изчислява печалбите и записва транзакциите по сигурен начин.

Етап 3 — Потребителски интерфейс на казиното (Седмици 5–6)

Цел: Създаване на първата напълно функционираща казино игра.

Седмица 5: Потребителски интерфейс

Задачи:

1. Проектиране на началната страница и навигацията на казиното (маршрутизацията с React Router е въведена в Седмица 2).
2. Изграждане на интерфейс на рулетката.
3. Добавяне на виртуални чипове и функционалност за избор на залог.
4. Визуализиране на наличния виртуален баланс.
5. Създаване на интерфейс за потвърждение на залог.
6. Свързване на действията в рулетката с backend API.
7. Визуализиране на печалби и загуби.
8. Обработка на API грешки и прекъснати връзки.

Седмица 6: Анимации и интеграция

Задачи:

1. Интегриране на Phaser за визуалните анимации.
2. Синхронизиране на анимацията на рулетката с резултатите от сървъра.
3. Добавяне на звукови ефекти, които могат да бъдат изключвани. Настройките се съхраняват в `user_preferences`.
4. Реализиране на история на игрите.
5. Добавяне на статистически панел.
6. Адаптиране на интерфейса за мобилни устройства.

7. Добавяне на настройка за намаляване на анимациите. Настройките се съхраняват в `user_preferences`.
8. Тестване на пълните игрови сценарии.

Очакван резултат: Интерактивна рулетка с реалистични правила, виртуална валута и визуални анимации.

Етап 4 — Образователно съдържание и награди (Седмици 7–8)

Цел: Реализиране на образователния компонент, който отличава платформата от стандартните безплатни казино симулатори.

Седмица 7: Библиотека с образователно съдържание

Задачи:

1. Проектиране на моделите за образователни модули и категории.
2. Добавяне на възможност потребителите да избират своите интереси.
3. Създаване на страница за разглеждане на образователно съдържание.
4. Интегриране на видео плейър за собствено или подходящо лицензирано съдържание.
5. Добавяне на описания и образователни цели.
6. Категоризиране на образователните материали.
7. Създаване на базова система за препоръки според интересите на потребителя.
8. Проследяване на завършените образователни дейности.

Седмица 8: Тестове и система за награди

Задачи:

1. Създаване на тестове с въпроси към образователните материали.
2. Реализиране на функционалност за изпращане на отговори.
3. Проверка на отговорите от backend системата.
4. Записване на резултатите от преминатите тестове.
5. Предоставяне на виртуални монети за допустими образователни дейности.
6. Предотвратяване на повторно получаване на една и съща награда.
7. Въвеждане на конфигурируем дневен лимит на наградите.
8. Визуализиране на образователния напредък и историята на наградите.
9. Добавяне на алтернативен безплатен начин за възстановяване на виртуалния баланс, който не изисква образователна дейност.

Важно е да се спазват условията на платформите за видео съдържание. YouTube видеа могат да бъдат използвани за обичайно образователно гледане, но не трябва да се предоставят награди, които стимулират гледането или зависят от възпроизвеждането на YouTube видеа.

Очакван резултат: Потребителите могат да разглеждат образователни материали, да решават тестове и да получават виртуални награди при предварително дефинирани условия.

Етап 5 — Потребителска статистика и отговорна игра (Седмици 9–10)

Цел: Добавяне на механизми за проследяване на активността, безопасност и управление на платформата.

Седмица 9: Потребителско табло и ограничения

Задачи:

1. Визуализиране на текущия виртуален баланс.
2. Показване на историята на транзакциите.
3. Показване на общия брой изиграни рундове.
4. Изчисляване на общите виртуални печалби и загуби.
5. Проследяване на времето, прекарано в игра.
6. Създаване на графики за игровата статистика.
7. Добавяне на информация за вероятностите и математическото предимство на казиното (House Edge).
8. Реализиране на доброволни ограничения за продължителността на игровите сесии.
9. Добавяне на възможност за временно спиране или самоизключване от играта.
10. Прилагане на ограниченията на backend ниво, а не само в потребителския интерфейс.

Седмица 10: Административен панел

Задачи:

1. Разработване на административен интерфейс.
2. Управление на образователните материали.
3. Управление на въпросите и тестовете.
4. Конфигуриране на виртуалните награди и дневните лимити.
5. Конфигуриране на игровите ограничения.
6. Преглед на обобщена статистика за платформата.
7. Преглед на одитни записи.
8. Защита на административните API endpoints чрез роли; деактивирането на потребител и смяната на роля задължително прекратяват активните му сесии.
9. Тестване на неоторизиран достъп и ограничения на сесиите.

Очакван резултат: Завършена MVP версия с казино симулация, образователно съдържание, потребителски статистики и административен панел.

Етап 6 — Тестване, хостинг и публикуване (Седмици 11–12)

Цел: Създаване на стабилна, сигурна и публично достъпна бета версия.

Седмица 11: Тестване и осигуряване на качеството

Задачи:

1. Завършване на backend Unit Tests и Integration Tests.
2. Изпълнение на frontend Component Tests.
3. Провеждане на End-to-End тестове.
4. Тестване на едновременни транзакции с виртуална валута.
5. Проверка на коректността при изчисляване на печалбите.
6. Тестване на прекъснати връзки, повторни заявки и дублирани операции.
7. Тестване на сигурността на автентикацията и авторизацията.
8. Проверка на използваните библиотеки и Docker изображения за уязвимости.
9. Тестване на достъпността и мобилната версия.
10. Проверка на процедурите за архивиране и възстановяване на данните.
11. Преглед на изискванията за поверителност, лицензиране и приложимите нормативни правила.
12. Проверка на съответствието между реализацията и API договора и на архитектурните правила.

Седмица 12: Deployment и представяне

Задачи:

1. Създаване на production Docker image за Spring Boot.
2. Публикуване на frontend приложението в Cloudflare Pages.
3. Публикуване на backend приложението в подходяща Docker хостинг среда.
4. Конфигуриране на управлявана PostgreSQL база данни.
5. Настройване на production environment variables.
6. Конфигуриране на HTTPS, маршрутизиране на API през същия сайт като frontend-а (заради блокирането на third-party cookies), `__Host-` cookies, доверени proxy сървъри, CORS и настройките за сигурност.
7. Изпълнение на database migrations.
8. Интегриране на мониторинг и обработка на грешки.
9. Извършване на финални Smoke Tests.
10. Подготовка на демонстрационен акаунт и сценарий за представяне.
11. Завършване на README файла и техническата документация.
12. Създаване на първата beta release версия в GitHub.

Очакван резултат: Напълно функциониращa и хостната бета версия на Online Casino Simulator.

---

5. Проектиране на базата данни

Таблица / Entity	Предназначение	Седмица
users	Потребителски акаунти, пароли, статуси и роли	2
interests	Каталог на образователните интереси	2
user_interests	Избраните от потребителя интереси	2
spring_session, spring_session_attributes	HTTP сесии (Spring Session JDBC)	2
auth_throttle	Ограничаване на опитите за вход и регистрация	2
user_preferences	Потребителски предпочитания за интерфейса (звук, анимации)	6
wallets	Текущ виртуален баланс и версия на записа	3
wallet_transactions	История на виртуалните транзакции	3
roulette_rounds	Рундове, резултати и времеви отметки	4
roulette_bets	Видове залози, суми и изплатени печалби	4
learning_modules	Образователни материали и категории	7
quiz_questions	Въпроси и правилни отговори	8
quiz_attempts	Отговори и резултати от тестовете	8
learning_rewards	Условия и получени награди	8
play_limits	Потребителски ограничения	9
user_sessions	История и продължителност на игровите сесии	9
audit_events	Важни системни и административни действия	10

Схемата се управлява изцяло чрез Flyway миграции. Към версия 1.1 на документа е **реализирана** само базовата миграция (V1); всички таблици по-горе са планирани и се създават от модула и в седмицата, които ги притежават.

Основни изисквания към базата данни

1. Потребителят не може да изразходва повече виртуални монети, отколкото притежава.

2. Един игрови рунд не може да бъде обработен и изплатен два пъти.
3. Една образователна награда не може да бъде получена повторно за същото допустимо изпълнение.
4. Приключените транзакции трябва да бъдат надеждно съхранявани и проследими.
5. Потребителската активност трябва да се записва последователно, като се съхраняват само необходимите лични данни.

---

6. Основни REST API endpoints

HTTP метод	Endpoint	Функционалност	Статус
GET	`/api/v1/system/status`	Състояние на платформата	**Реализиран**
GET	`/api/v1/auth/csrf`	Получаване на CSRF токен	Седмица 2
POST	`/api/v1/auth/register`	Регистрация	Седмица 2
POST	`/api/v1/auth/login`	Вход	Седмица 2
POST	`/api/v1/auth/logout`	Изход	Седмица 2
GET	`/api/v1/users/me`	Извличане на потребителски профил	Седмица 2
PATCH	`/api/v1/users/me`	Редактиране на профил	Седмица 2
GET	`/api/v1/interests`	Каталог на интересите	Седмица 2
GET	`/api/v1/wallet`	Текущ виртуален баланс	Седмица 3
GET	`/api/v1/wallet/transactions`	История на транзакциите	Седмица 3
POST	`/api/v1/games/roulette/spins`	Поставяне на залог и обработка на игрови рунд	Седмица 4
GET	`/api/v1/games/roulette/history`	История на рулетката	Седмица 6

HTTP метод	Endpoint	Функционалност	Статус
GET	`/api/v1/learning/modules`	Образователни материали	Седмица 7
POST	`/api/v1/learning/modules/{id}/attempts`	Изпращане на отговори на тест	Седмица 8
POST	`/api/v1/learning/modules/{id}/rewards`	Получаване на допустима награда	Седмица 8
GET	`/api/v1/dashboard`	Потребителска статистика	Седмица 9
PUT	`/api/v1/play-limits`	Настройване на игрови ограничения	Седмица 9

Административните API endpoints ще използват префикса `/api/v1/admin/`.

Всички endpoints трябва да валидират входните данни и да прилагат изискванията за авторизация и бизнес логика на сървърно ниво.

Пълната спецификация на API се поддържа във файла `openapi.yaml` (design-first), който е източникът на истина за всички endpoints. Нов endpoint се описва първо в договора и едва след това се реализира; автоматични тестове проверяват съответствието.

---

7. Тестване и критерии за приемане

MVP версията ще се счита за успешно разработена, когато са изпълнени следните критерии:

Компонент	Критерий за приемане
Автентикация	Защитените endpoints не допускат неоторизирани заявки
Виртуален портфейл	Не са възможни отрицателни баланси или дублирани транзакции
Рулетка	Всички 37 резултата имат еднаква вероятност
Изчисляване на печалби	Печалбите съответстват на стандартните правила

Компонент	Критерий за приемане
Цялост на данните	Обработката на игровия рунд и обновяването на баланса са атомарни
Образователен модул	Наградите се предоставят само при валидно изпълнение и не се дублират
Игрови ограничения	Почивките и самоизключването блокират нови игрови действия
Статистика	Показателите съответстват на записаните транзакции
Frontend	Основните функционалности работят на компютър и мобилно устройство
Deployment	Всяка нова версия преминава автоматизираните проверки и Smoke Tests
API договор	Реализацията съответства на `openapi.yaml` (автоматично проверено)
Архитектура	Модулните граници и слоевете се спазват (ArchUnit)

Препоръчителни показатели за качество

- Минимум 80% Unit Test Coverage за критичната бизнес логика на портфейла и игровия engine.
- Автоматизирани тестове за всички важни правила за достъп.
- Многократни тестове на едновременни транзакции.
- Статистическа проверка на случайността на резултатите, отделно от Unit Tests.
- Липса на известни критични уязвимости при публикуването на бета версията.
- Успешно преминаване на End-to-End тестове за основния потребителски сценарий.

---

8. Структура на проекта

Организация на файловете и директориите (хранилище `online-casino-simulator`):

```
online-casino-simulator/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── games/
│   │   ├── features/
│   │   ├── services/
│   │   └── types/
│   └── package.json
│
├── backend/
│   ├── src/main/java/com/casino/
│   │   ├── auth/  user/  wallet/  game/
│   │   ├── learning/  responsibleplay/  admin/
│   │   │   └── controller/  service/  repository/
│   │   │       entity/  dto/  config/  exception/
│   │   └── common/   (config/, exception/, system/)
│   ├── src/main/resources/
│   │   ├── db/migration/
│   │   └── static/openapi.yaml
│   └── pom.xml
│
├── infrastructure/
│   ├── docker/
│   └── deployment/
│
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── database.md
│   ├── user-scenarios.md
│   ├── decisions/
│   ├── plans/
│   └── proposals/
│
├── .github/workflows/
├── docker-compose.yml
├── .env.example
└── README.md
```

Слоевите пакети на бизнес модулите се създават при реализацията им; към версия 1.1 е реализиран модулът `common`.

---

9. Организация на разработката

За управление на задачите ще бъдат използвани GitHub Issues и GitHub Projects.

Препоръчителни статуси на задачите

Backlog → Ready → In Progress → In Review → Testing → Done

Работен процес при разработване на функционалност

1. Дефиниране на функционалните изисквания и критериите за приемане.
2. Проектиране на необходимите промени в базата данни.
3. Проектиране или обновяване на API договора (`openapi.yaml`) и неговото одобрение.
4. Реализиране на backend логиката и тестовете спрямо договора.
5. Разработване на съответните API endpoints.
6. Създаване на frontend интерфейса.
7. Добавяне на Integration или End-to-End тестове.
8. Проверка на сигурността и влиянието върху отговорната игра.
9. Преглед и обединяване на кода чрез Pull Request.
10. Проверка на функционалността в средата за разработка.

Препоръчва се използване на кратки feature branches, автоматизирани проверки на Pull Requests и редовно представяне на напредъка пред ментора.

---

10. Основни рискове и решения

Риск 1: Прекалено голям обхват на проекта

Решение: Първоначално ще бъде разработена само рулетка. Останалите игри ще бъдат добавени в следващи версии.

Риск 2: Некоректни виртуални баланси

Решение: Използване на атомарни транзакции, idempotency механизми и тестове за едновременни заявки.

Риск 3: Злоупотреба със системата за образователни награди

Решение: Проверка на резултатите на сървъра, уникални заявки за награди и дневни лимити.

Риск 4: Нарушаване на условията за използване на външно видео съдържание

Решение: Използване на собствено, подходящо лицензирано или разрешено за съответната употреба образователно съдържание.

Риск 5: Негативно влияние върху потребители с хазартни проблеми

Решение: Прозрачна информация за вероятностите, реални ограничения на сесиите и независима оценка на безопасността на продукта.

Риск 6: Ограничения на безплатните хостинг платформи

Решение: Използване на Docker за преносимост и документиране на алтернативен начин за deployment.

Риск 7: Уязвимости в сигурността

Решение: Защитени session cookies (HttpOnly; Secure и `__Host-` в production), CSRF защита, ограничаване на опитите за вход, сървърна валидация, автоматизирани тестове и проверка на зависимостите.

Риск 8: Правни изисквания и защита на личните данни

Решение: Преглед на приложимото законодателство, правилата за симулирани хазартни игри и изискванията за защита на личните данни преди публичното пускане.

---

11. Бъдещо развитие

След успешното завършване на MVP могат да бъдат разработени следните версии.

Версия 1.1 — Допълнителни казино игри

- Blackjack.
- Slot Machines с документирани математически модели.
- Допълнителни варианти на рулетка.

Версия 1.2 — Разширен образователен модул

- Повече образователни категории.
- Персонализирани препоръки според интересите на потребителя.
- Табло за проследяване на образователния напредък.
- Персонализирани образователни пътеки.

Версия 1.3 — Разширени технически функционалности

- WebSocket комуникация за събития в реално време, когато има реална необходимост.
- Разширен мониторинг и анализ на производителността.
- Load Testing и оптимизации за по-голям брой потребители.
- Публична документация на математическите модели на игрите и възможност за независима проверка на статистическата им коректност.

Всички бъдещи функционалности трябва да бъдат оценявани както от техническа гледна точка, така и по отношение на безопасността на потребителите.

---

12. Крайни резултати от проекта

След приключване на разработката проектът трябва да включва:

1. Напълно функциониращо full-stack уеб приложение.
2. Поне една завършена казино игра.
3. Надежден виртуален портфейл с безопасна обработка на транзакциите.
4. Образователен модул с виртуални награди.
5. Потребителски профил и статистическо табло.
6. Административен панел.
7. Функционалности за отговорна игра.
8. Автоматизирани frontend и backend тестове.
9. Docker среда за локална разработка.
10. Публично достъпна бета версия.
11. Изходен код с ясна техническа документация.
12. Демонстрация на продукта и представяне на архитектурата.

Критерий за завършен проект

Проектът ще се счита за функционално завършен, когато потребител може:

- Да създаде акаунт.
- Да получи начална виртуална валута.
- Да играе рулетка с математически коректни резултати.
- Да преглежда историята на игрите и транзакциите си.
- Да използва образователното съдържание.
- Да получава допустими виртуални награди.
- Да настройва ограничения за игровите сесии.
- Да излезе от системата и впоследствие да продължи използването ѝ, без да губи информацията за своя профил и транзакции.

---

13. Обобщен график и основни етапи

Седмици 1–2: Основи на проекта

Резултат: Настроена среда за разработка, база данни и функционираща автентикация. **(Седмица 1 — завършена)**

Седмици 3–4: Виртуален портфейл и логика на рулетката

Резултат: Сигурно управление на виртуалния баланс и тествана логика на играта.

Седмици 5–6: Интерактивно казино

Резултат: Функционираща рулетка с потребителски интерфейс, свързан с backend системата.

Седмици 7–8: Образователно съдържание и награди

Резултат: Образователни модули, тестове и система за виртуални награди.

Седмици 9–10: Статистика и административни функции

Резултат: Потребителско табло, административен панел и механизми за отговорна игра.

Седмици 11–12: Тестване и публикуване

Резултат: Тествана, документирана и публикувана бета версия на приложението.

---

14. Основен технически приоритет

Най-важният технически етап е приключването на четвъртата седмица.

До този момент виртуалният портфейл и логиката на рулетката трябва да функционират коректно чрез API заявки и автоматизирани тестове, дори без наличието на графичен интерфейс.

Това е основата, върху която ще бъдат изградени всички следващи функционалности.

При представянето на проекта пред SAP е важно да бъдат демонстрирани не само визуалният интерфейс и игровото преживяване, но и качеството на софтуерната реализация:

- Архитектурата на приложението.
- Организацията на backend модулите.
- Управлението на базата данни.
- Надеждността на транзакциите.
- Коректността на игровите алгоритми.

- Сигурността на системата.
- Автоматизираното тестване.
- Използването на Docker.
- Процесът на публикуване и поддръжка.

Крайната цел е разработването на качествен, завършен и технически добре структуриран софтуерен продукт, който демонстрира практически умения в създаването на съвременни full-stack приложения.

Приложение А. История на версиите на документа

Номерата се отнасят до версиите на този документ, а не до продуктовите версии в раздел 11.

Версия	Дата	Описание
1.0	23.10.2026	Първоначален план за разработка (дата на оригиналния файл).
1.1	29.10.2026	Актуализация на документа спрямо приетите архитектурни решения (ADR-0002, ADR-0004, ADR-0005), предложеното решение за сигурност (ADR-0006) и плана за Седмица 2 (ревизия 3). Графикът от 12 седмици, целите и обхватът на MVP са непроменени.

Промени във версия 1.1

№	Раздел	Промяна
C1	2. Използвани технологии	Колона „Статус“; JUnit Jupiter (JUnit 6) вместо JUnit 5; нови редове: React Router, Spring Session JDBC, ArchUnit; OpenAPI като design-first договор.
C2	3. Архитектура	Нова подточка 3.2.1: слоеста структура на модулите и API договор.
C3	4. Седмица 2	Задачи 11–15: API договор първо, CSRF, ограничаване на опитите за вход, ограничение на сесиите, архитектурни и договорни тестове.
C4	4. Седмица 5	Навигацията надгражда маршрутизацията от Седмица 2.
C5	4. Седмица 6	Настройките за звук и анимации се съхраняват в `user_preferences`.
C6	4. Седмица 10	Деактивирането на потребител и смяната на роля задължително прекратяват сесиите му.
C7	4. Седмица 11	Нова задача 12: проверка на API договора и архитектурните правила.
C8	4. Седмица 12	Маршрутизиране на API през същия сайт, `__Host-` cookies, доверени proxy сървъри.

№	Раздел	Промяна
C9	5. База данни	Колона „Седмица“; нови таблици `interests`, `user_interests`, `spring_session*`, `auth_throttle`; уточнено значение на `user_preferences` и `user_sessions`.
C10	6. REST API	Колона „Статус“; нови endpoints `/system/status`, `/auth/csrf`, `/interests`; препратка към `openapi.yaml`.
C11	7. Критерии за приемане	Нови критерии: API договор и архитектура.
C12	8. Структура на проекта	Пакети в модулите, `openapi.yaml`, `docs/plans`, `docs/proposals`, име на хранилището.
C13	9. Работен процес	Нова стъпка 3: проектиране и одобрение на API договора.
C14	3.4 и 10. Риск 7	Кратко описание на мерките за сигурност (планирано за Седмица 2).
C15	Заглавна част, 4, 13	Версия на документа, означения „Реализирано“/ „Планирано“, статус на Седмици 1 и 2.

Подробно описание на промените и мотивите: `docs/proposals/0001-implementation-plan-alignment.md`. Версия 1.0 на документа се запазва непроменена в хранилището.